LAB 06 Scrollable lists, cards, search and user feedback

Unit 3 • 3-hour guided laboratory • CLO-5

Learning goal. Render large item collections efficiently, filter results and manage deletion through dialogs and undo patterns.

Core concepts. ListView.builder, Card, ListTile, Dismissible, confirm dialogs, SnackBarAction, controller cleanup, stable keys.

Before you start. Create a separate project for each solved task or replace lib/main.dart between executions.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab06
cd mad_lab06
```

Solved task 1 | Searchable noticeboard with lazy list items

Problem and objective. Simulate 200 announcements and filter them by their title. Tapping a tile should show the selected announcement in a SnackBar.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: NoticePage()));

class NoticePage extends StatefulWidget {
  const NoticePage({super.key});
  @override
  State<NoticePage> createState() => _NoticePageState();
}

class _NoticePageState extends State<NoticePage> {
  final search = TextEditingController();
  final notices = List.generate(200, (i) => 'Campus notice ${i + 1}');
  @override
  void dispose() { search.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) {
    final visible = notices.where((e) =>
      e.toLowerCase().contains(search.text.toLowerCase())).toList();
    return Scaffold(
      appBar: AppBar(title: const Text('Campus notices')),
      body: Column(children: [
        Padding(padding: const EdgeInsets.all(12), child: TextField(
          controller: search,
          decoration: const InputDecoration(
            prefixIcon: Icon(Icons.search), labelText: 'Search notices',
            border: OutlineInputBorder(),
          ),
          onChanged: (_) => setState(() {}),
        )),
        Expanded(child: ListView.builder(
          itemCount: visible.length,
          itemBuilder: (context, i) => Card(
            child: ListTile(
              leading: const Icon(Icons.campaign_outlined),
              title: Text(visible[i]),
              subtitle: const Text('COMSATS Sahiwal'),
              onTap: () => ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(content: Text('Opened ${visible[i]}'))),
            ),
          ),
        )),
      ]),
    );
  }
}
```

How the solution works. List.generate provides sample records; filtering occurs each rebuild, sufficient for 200 values. ListView.builder creates visible rows on demand and only one scrollable remains in the available Expanded area.

Expected output. The student sees 200 entries, can search 18 or another number, and receives a SnackBar on tap.

Check your work. Search for 999; the list should show zero rows without an exception.

Solved task 2 | Dismissible task list with confirmation and undo

Problem and objective. Create a list where swipe-to-delete requests confirmation, then offers a reversible deletion.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: SwipePage()));

class SwipePage extends StatefulWidget {
  const SwipePage({super.key});
  @override
  State<SwipePage> createState() => _SwipePageState();
}

class _SwipePageState extends State<SwipePage> {
  final List<String> tasks = ['Read Flutter docs',
    'Finish layout lab', 'Review pull request', 'Run widget tests'];

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Swipe tasks')),
    body: ListView.builder(
      itemCount: tasks.length,
      itemBuilder: (context, i) {
        final item = tasks[i];
        return Dismissible(
          key: ValueKey(item),
          background: Container(color: Colors.red.shade100,
            alignment: Alignment.centerLeft,
            child: const Icon(Icons.delete_outline)),
          confirmDismiss: (_) async => await showDialog<bool>(
            context: context,
            builder: (dialogContext) => AlertDialog(
              title: const Text('Delete task?'),
              content: Text(item),
              actions: [
                TextButton(onPressed: () => Navigator.pop(dialogContext, false),
                    child: const Text('Cancel')),
                FilledButton(onPressed: () => Navigator.pop(dialogContext, true),
                    child: const Text('Delete')),
              ],
            ),
          ) ?? false,
          onDismissed: (_) {
            setState(() => tasks.remove(item));
            ScaffoldMessenger.of(context).hideCurrentSnackBar();
            ScaffoldMessenger.of(context).showSnackBar(SnackBar(
              content: Text('Deleted $item'),
              action: SnackBarAction(label: 'UNDO', onPressed: () {
                setState(() => tasks.insert((i < tasks.length ? i : tasks.length), item));
              }),
            ));
          },
          child: ListTile(title: Text(item)),
        );
      },
    ),
  );
}
```

How the solution works. Dismissible needs a stable identity key so Flutter can reconcile reordered or removed rows. confirmDismiss waits for user consent before moving the item. SnackBarAction inserts the removed value back into the list.

Expected output. Swiping produces a confirmation; Delete removes the row and Undo restores it.

Check your work. Dismiss the second item and tap Undo. Explain why list index alone is an unsafe key after deletion.

Scenario-based practice — unsolved

Scenario 1. Large library catalog. Build a 5,000-book catalog with search and category filters. Explain how you would avoid rebuilding every tile and debounce expensive searches.

Scenario 2. Student attendance. Allow swiping absent records with a confirmation and undo. Explain what server/database changes are required for a truly reliable undo.

Technical viva — justify your reasoning

1. Why does a Dismissible widget require a key?
2. What rebuild cost does ListView.builder save relative to Column with all children?
3. What data consistency issue arises if Undo is offered after a server delete?
4. When is an AlertDialog preferable to a SnackBar?
5. What happens to list state when the app process is killed?
6. How can stable unique IDs prevent state mixing between rows?

Submission and self-check

Provide search screenshots, successful undo demonstration, and an explanation of identity keys.